



資料結構

Data Structure

Lab 9

姓名： 陳冠豪

學號： 114310125

Lab09-Q1

Delete Node in a Binary Search Tree

Code

```
#include <iostream>
using namespace std;

// 樹的節點類別
class TreeNode {
public:
    int data;           // 這個節點存的數值
    TreeNode* left;     // 指向左邊子節點
    TreeNode* right;    // 指向右邊子節點

    // 建構子：建立節點時會設定數值，左右先設為空
    TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
};

// 二元搜尋樹類別
class BinarySearchTree {
private:
    TreeNode* root; // 樹的最上面(根節點)

    // 遞迴插入
    TreeNode* insert(TreeNode* node, int val) {
        if (node == nullptr) { // 如果目前位置是空的
            return new TreeNode(val); // 就在這裡建立新節點
        }

        if (val < node->data) { // 如果值比較小
            node->left = insert(node->left, val); // 往左邊插
        }
        else if (val > node->data) { // 如果值比較大
            node->right = insert(node->right, val); // 往右邊插
        }
    }
};
```

```

    }
    //如果一樣就不做事(避免重複)

    return node; // 回傳目前節點 (讓上一層接回去)
}

//遞迴搜尋
bool search(TreeNode* node, int target) {
    if (node == nullptr) return false; //找到空還沒找到=不存在

    if (node->data == target) return true; //找到了
    else if (target < node->data) //比目前小=往左找
        return search(node->left, target);
    else //比目前大=往右找
        return search(node->right, target);
}

//中序遍歷(左→根→右)
void inOrderTraversal(TreeNode* node) {
    if (node == nullptr) return; //沒東西就直接回去

    inOrderTraversal(node->left); //先走左邊
    cout << node->data << " "; //印出自己
    inOrderTraversal(node->right); //再走右邊
}

//刪除節點
TreeNode* deleteNode(TreeNode* node, int val) {
    if (node == nullptr) return node; //找不到就直接回傳

    if (val < node->data) { //要刪的值比較小=往左找
        node->left = deleteNode(node->left, val);
    }
    else if (val > node->data) { //比較大=往右找
        node->right = deleteNode(node->right, val);
    }
    else { //找到了要刪的節點

```

```

        //情況 1：沒有左子樹(或只有右邊)
        if (node->left == nullptr) {
            TreeNode* temp = node->right; //暫存右子樹
            delete node;                  //刪掉自己
            return temp;                  //用右子樹補上
        }
        //情況 2：沒有右子樹(或只有左邊)
        else if (node->right == nullptr) {
            TreeNode* temp = node->left; //暫存左子樹
            delete node;                  //刪掉自己
            return temp;                  //用左子樹補上
        }

        //情況 3：左右都有子樹
        //找右子樹中最小的(當替代者)
        TreeNode* temp = findMin(node->right);

        node->data = temp->data; //用那個最小值取代目前節點

        //再去右子樹把剛剛用掉的那個刪掉
        node->right = deleteNode(node->right, temp->data);
    }

    return node; //回傳更新後的節點
}

//找最小值
TreeNode* findMin(TreeNode* node) {
    while (node->left != nullptr) { //一直往左走
        node = node->left;
    }
    return node; //最左邊就是最小
}

public:
    //建構子:一開始是空樹
    BinarySearchTree() : root(nullptr) {}

```

```

//對外的插入(從 root 開始)
void insert(int val) {
    root = insert(root, val);
}

//對外的搜尋
bool search(int target) {
    return search(root, target);
}

//印出排序結果(中序)
void displayInOrder() {
    cout << "樹的中序遍歷(排序後)結果：" << endl;
    inOrderTraversal(root);
    cout << endl;
}

//對外刪除
void deleteValue(int val) {
    root = deleteNode(root, val);
}
};

int main() {
    BinarySearchTree bst; //建立一棵空樹

    //預設資料
    int values[] = { 7, 1, 4, 2, 8, 13, 12, 11, 15, 9, 5 };
    int n = sizeof(values) / sizeof(values[0]); // 算有幾個數字

    //一個一個插進樹裡
    for (int i = 0; i < n; ++i) {
        bst.insert(values[i]);
    }

    int key; //使用者輸入的數字

    cout << "請輸入要搜尋/新增的鍵值：";

```

```

cin >> key;

if (bst.search(key)) { //如果找得到
    cout << "值 " << key << " 已存在樹中，不新增。" << endl;
}
else { //找不到就加進去
    cout << "值 " << key << " 不存在，已新增進樹中。" << endl;
    bst.insert(key);
}

//顯示排序後結果
bst.displayInOrder();

int deleteKey;

cout << "請輸入要刪除的鍵值：" << endl;
cin >> deleteKey;

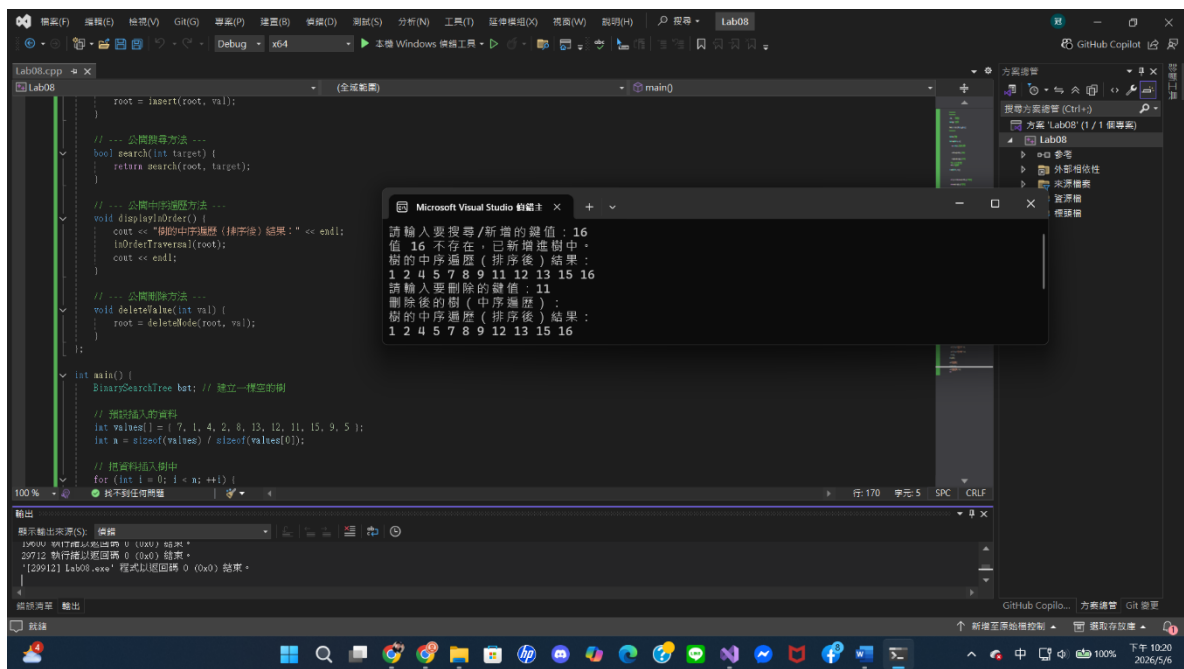
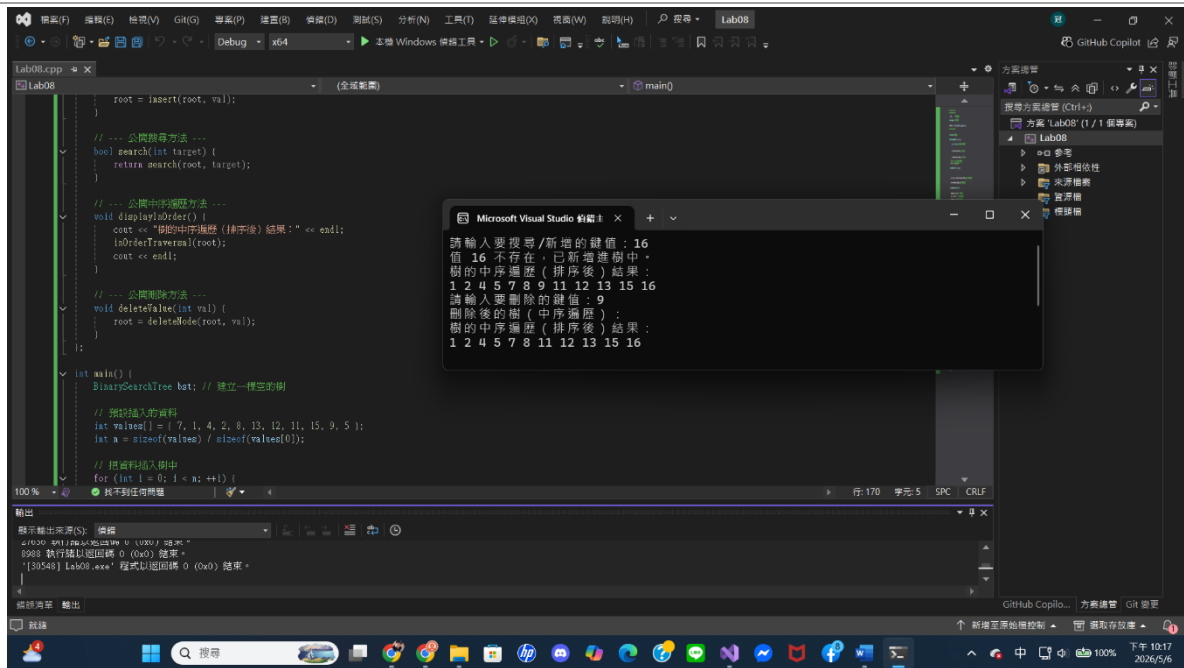
bst.deleteValue(deleteKey); //刪掉

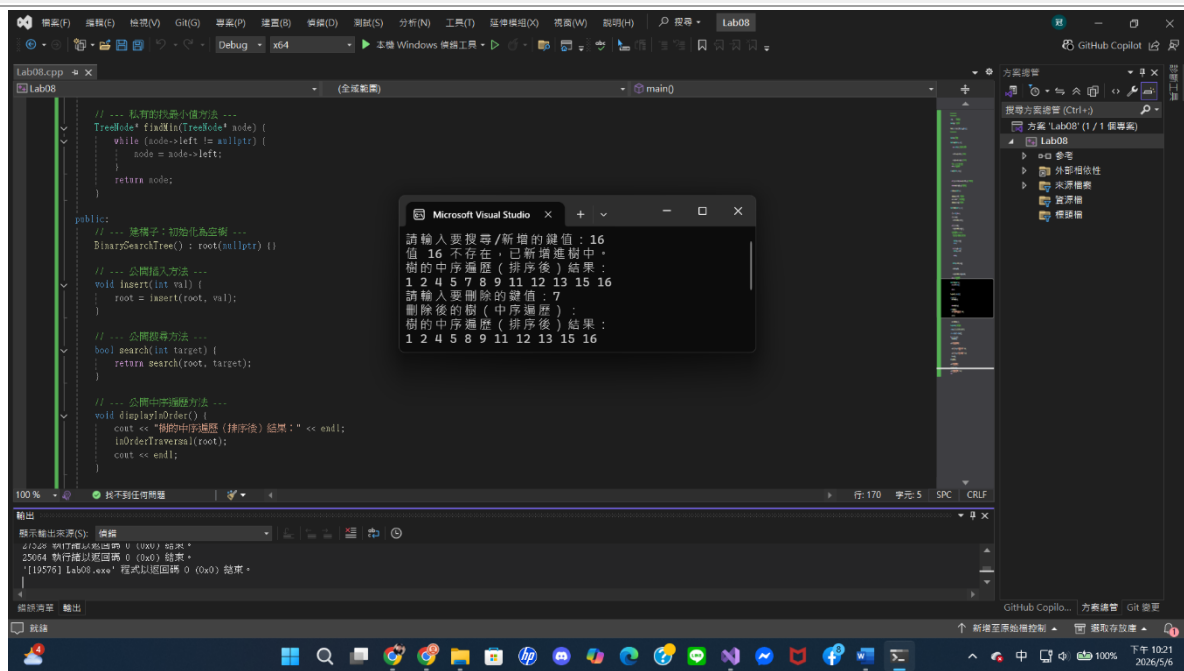
cout << "刪除後的樹（中序遍歷）：" << endl;
bst.displayInOrder(); //再印一次

return 0;
}

```

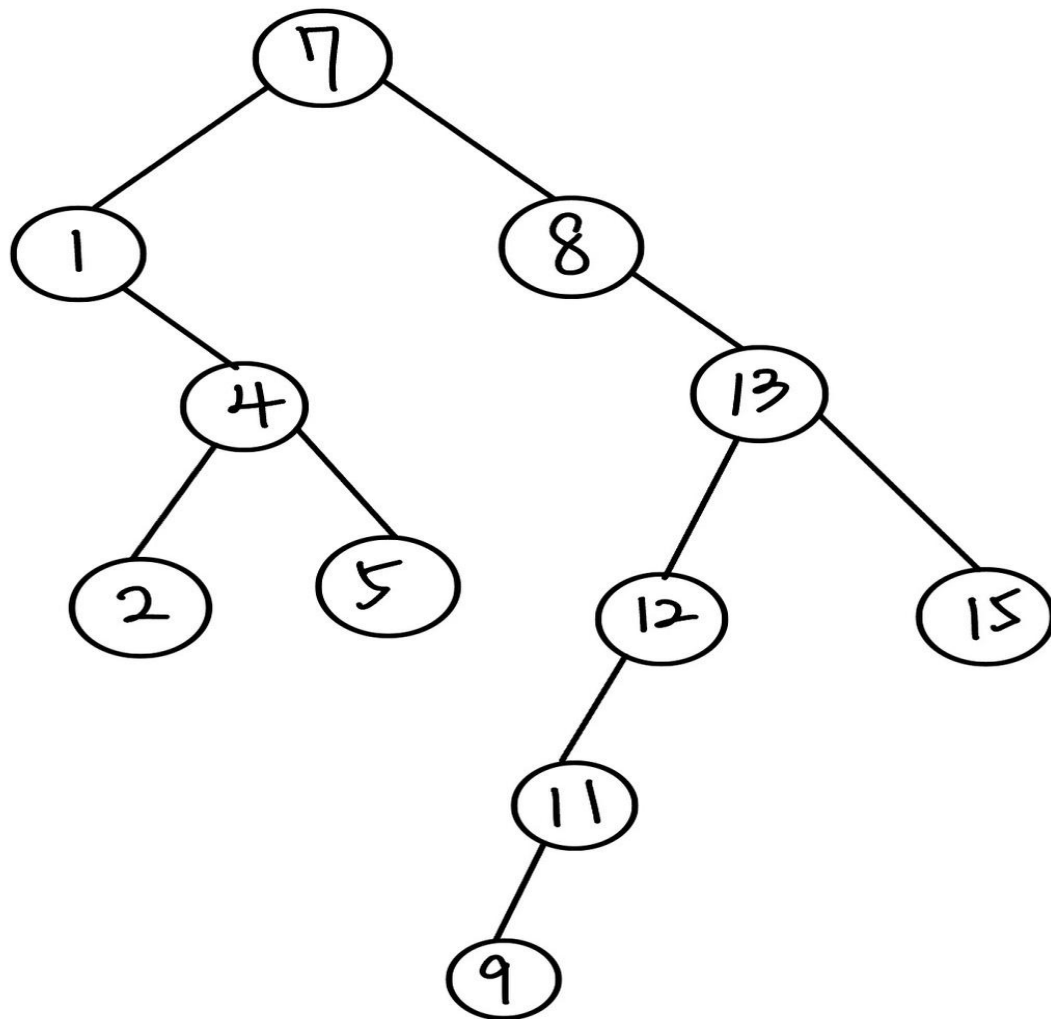
Result



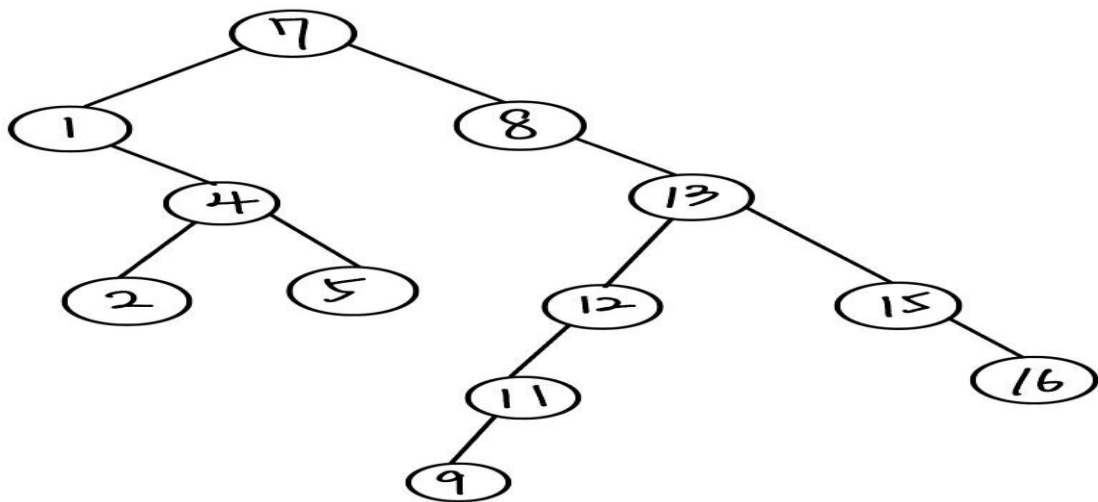


Discussion

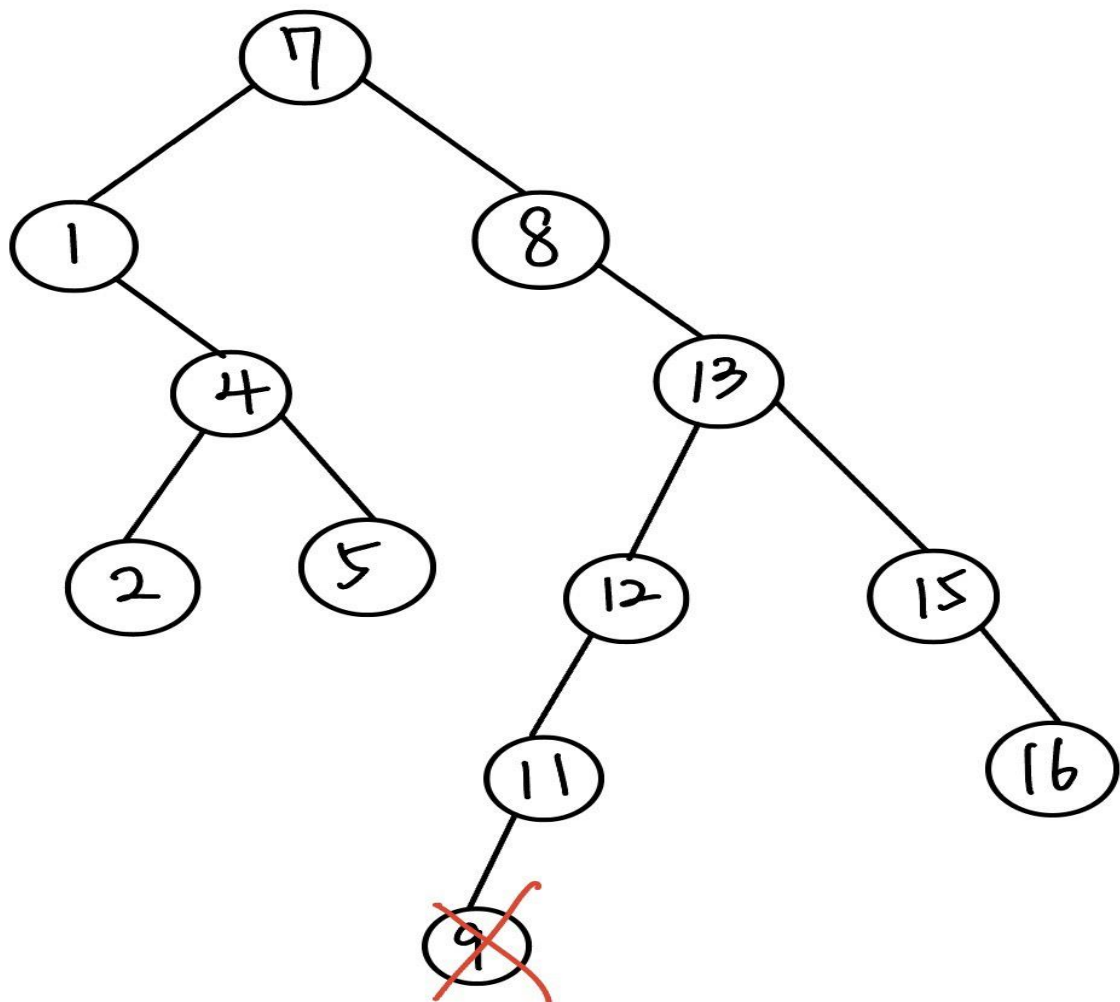
初始狀態:



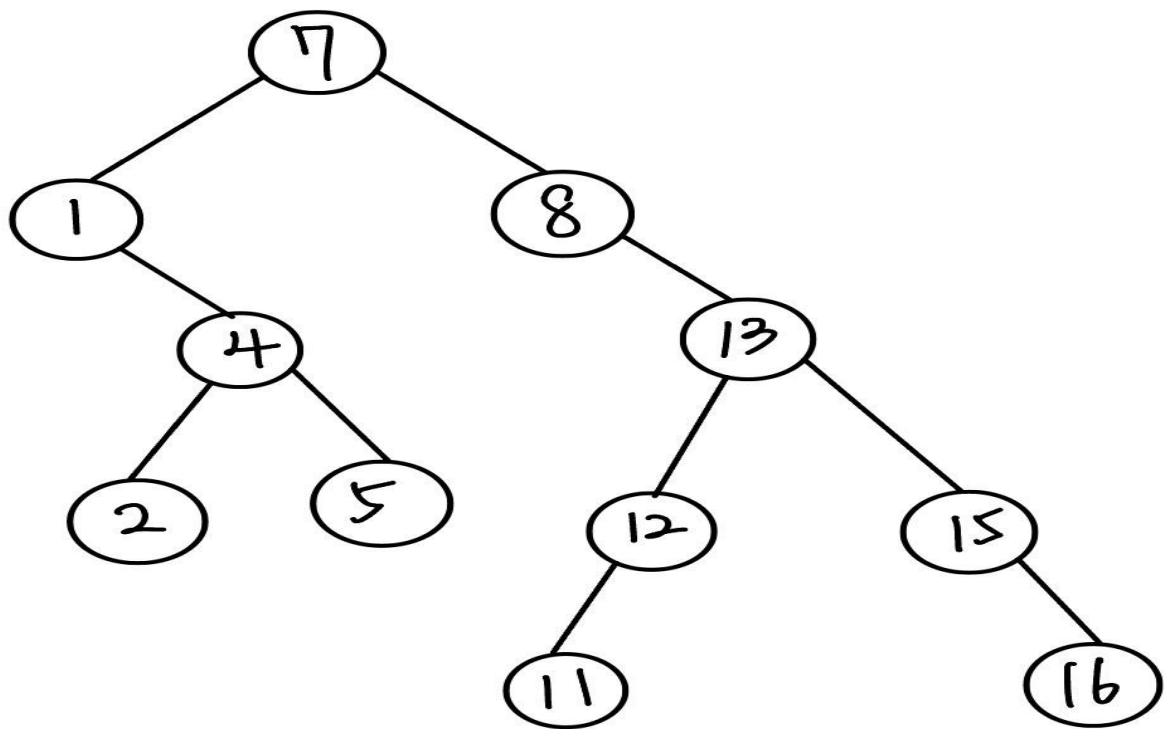
插入 16



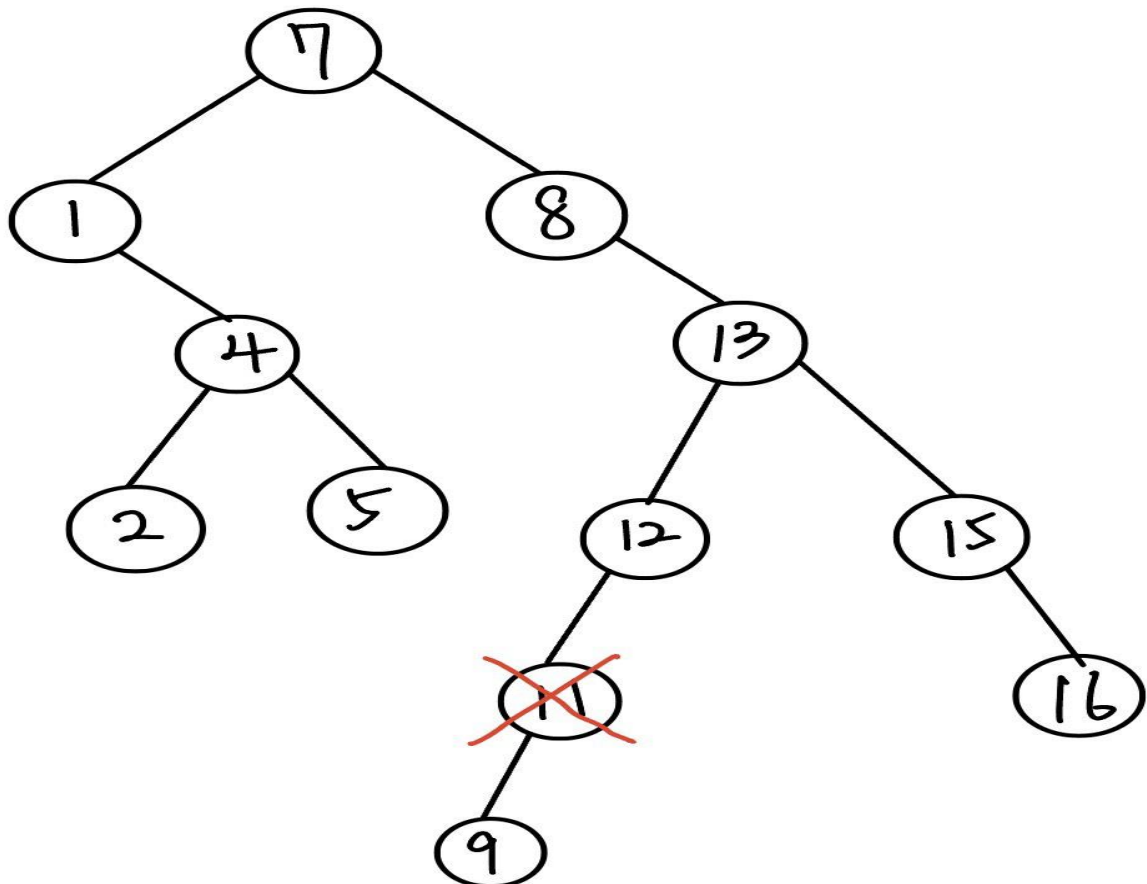
刪除的第一種情況：目標節點完全沒有子節點(目標節點為 9)



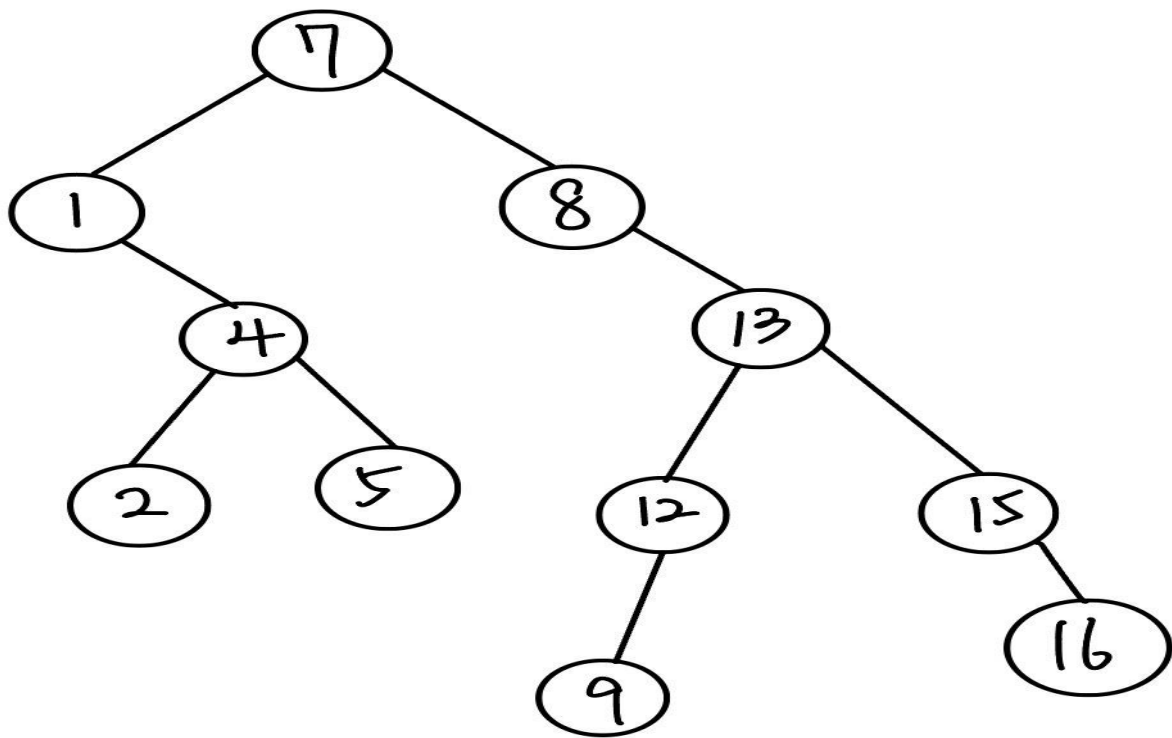
刪除後狀態



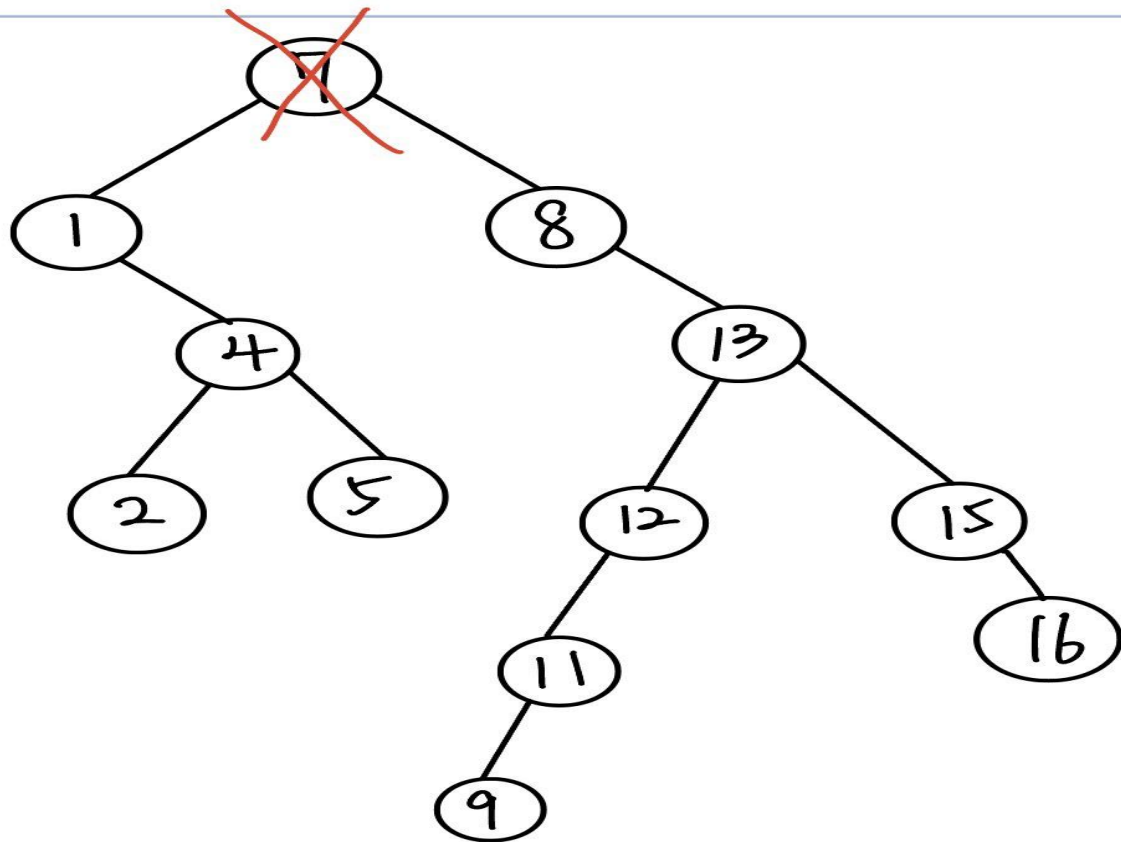
刪除的第二種情況：目標節點只有左邊或右邊有子節點(目標節點為 11)



刪除後狀態



刪除的第三種情況：目標節點兩邊都有子節點(目標節點為 7)



刪除後狀態

